RoundFi

Architecture Specification

Solana-Native Behavioral Credit Infrastructure

Version 1.0 - May 2026

Hackathon-stage architecture specification

Solana Devnet - Open-source protocol

Architecture Specification

Solana-Native Behavioral Credit Infrastructure

Version 1.0 - May 2026 - Hackathon-stage architecture specification

Core architecture: Solana programs, deterministic PDAs, reputation records, yield adapters, SDK, front-end, Stress Lab, and future B2B oracle surfaces working toward a portable behavioral credit layer.

Executive Summary

RoundFi is a Solana-native behavioral credit protocol that turns recurring payments into portable on-chain reputation.

This document explains the architecture behind that thesis.

RoundFi is composed of on-chain Anchor programs, deterministic PDAs, Metaplex Core position assets, a reputation layer, yield adapters, a TypeScript SDK, a Next.js front-end, a Stress Lab economic simulator, and an indexer scaffold.

The product is not only a savings group application.

The savings group is the behavior-generation mechanism.

The architecture is designed to produce a reusable behavioral credit signal that can eventually be consumed by DeFi protocols, fintechs, neobanks, and credit applications.

The current implementation is hackathon-stage and validated on Solana Devnet. It includes a functional prototype, live front-end, four deployed program surfaces, devnet pools, app-to-chain flows for selected actions, Stress Lab simulations, internal security review, and open-source code.

Mainnet deployment, external audit, production indexer, production SAS issuance, and B2B oracle API remain roadmap items.

1. Architectural Thesis

RoundFi is designed around one core thesis:

Recurring financial behavior can become portable collateral intelligence.

Most DeFi lending systems protect themselves through overcollateralization. This is effective for protocol safety, but it is inefficient for users. The system mostly asks: how much capital can this user lock?

RoundFi asks a different question:

Can this user consistently meet a recurring financial commitment over time?

The architecture exists to generate, verify, store, and expose that signal.

At a high level:

```
Savings group participation
  -> recurring payments
  -> on-chain records / attestations
```

```
-> behavioral reputation
-> dynamic collateral reduction
-> future B2B score oracle
```

The core architecture separates the system into independent surfaces so that pool mechanics, reputation, yield, indexing, and user experience can evolve without breaking the entire protocol.

2. System Goals

The RoundFi architecture has seven primary goals.

1. Generate behavioral credit data on-chain

Users must earn reputation through real recurring participation, not through arbitrary profile claims.

2. Reduce collateral requirements over time

The reputation ladder allows users to progress from 50% to 30% to 10% collateral/stake requirements as they prove behavior.

3. Keep economic risk bounded

The Triple Shield model, escrow, stake, solidarity vault, and Stress Lab exist to keep losses bounded within modeled conditions.

4. Make reputation portable

Reputation should not be trapped inside the front-end. It should be readable by wallets, protocols, and future APIs.

5. Use Solana-native composability

RoundFi is designed for frequent low-cost transactions, wallet flows, CPIs, Metaplex assets, yield adapters, and future attestations.

6. Separate shipped, under-verification, and roadmap

The architecture must remain honest about what is live on devnet, what is implemented but needs verification, and what is still future work.

7. Support a B2B reputation oracle endgame

The protocol starts with groups, but the long-term business model is reputation infrastructure.

3. Non-Goals for the Current Phase

The current hackathon-stage implementation is not intended to be a fully regulated production credit platform.

Current non-goals include:

- no mainnet credit deployment before external audit;
- no claim of production-grade underwriting;
- no guarantee that defaults cannot happen;
- no production B2B oracle yet;
- no hosted production indexer yet;
- no fully externalized SAS production issuance yet;

- no fiat on-ramp requirement in the core architecture;
- no governance token in the current phase;
- no cross-chain bridge dependency.

The current goal is to prove the protocol mechanism, architecture, devnet execution, economic model, and team execution.

4. High-Level Architecture

RoundFi is composed of seven major layers.

Layer	Function	Current Status
Core protocol	Pool lifecycle, contributions, payouts, escrow, default settlement, escape valve, yield routing	Devnet deployed
Reputation program	Reputation profiles, SAS-compatible attestations, score mechanics	Devnet deployed / integration in progress
Yield adapters	Mock yield adapter for devnet and Kamino-oriented adapter surface	Deployed surfaces / production use roadmap
Position assets	Metaplex Core position NFTs for member positions and escape valve transfers	Implemented / devnet exercised
SDK	IDL-free TypeScript helpers, account decoders, PDA helpers, event utilities	Shipped
Front-end app	User dashboard, groups, wallet, reputation, market, Stress Lab, demo studio	Live prototype
Indexer / API foundation	Event ingestion, Helius webhook, Prisma/Postgres schema, future score API foundation	Scaffold shipped / roadmap for production

The simplified architecture is:

```
User wallet
  -> Front-end app
  -> TypeScript SDK
  -> Solana programs
     -> roundfi-core
     -> roundfi-reputation
     -> yield adapter
     -> Metaplex Core
  -> Indexer / API layer
  -> Future B2B reputation oracle
```

5. Program Topology

RoundFi uses a modular on-chain design.

The repository architecture describes a protocol centered around roundfi-core, supported by separate reputation and yield adapter programs.

```
roundfi-core
  |-- CPI -> roundfi-reputation
  |-- CPI -> Metaplex Core
```

```
|-- CPI -> yield adapter interface
        |-- roundfi-yield-mock
        |-- roundfi-yield-kamino
```

5.1 `roundfi-core`

roundfi-core is the main state machine.

It is responsible for:

- protocol initialization;
- pool creation;
- member joining;
- stake management;
- recurring contributions;
- payout claims;
- escrow accounting;
- default settlement;
- escape valve listing and buying;
- yield deposit and harvest;
- pool closing;
- pause and treasury controls.

This program contains the most important economic logic of the protocol.

5.2 `roundfi-reputation`

roundfi-reputation manages wallet-level behavioral profiles and attestation records.

It is responsible for:

- reputation profiles;
- payment records;
- late/default records;
- cycle-completion records;
- score updates;
- level promotion;
- identity hints;
- future SAS-compatible reputation surfaces.

This program is separated from roundfi-core so the reputation layer can become a reusable primitive.

5.3 `roundfi-yield-mock`

roundfi-yield-mock exists for devnet and simulation.

It allows the protocol to test yield routing without requiring production yield assumptions.

5.4 `roundfi-yield-kamino`

roundfi-yield-kamino is the Kamino-oriented yield adapter surface.

Its purpose is to let the protocol interact with Solana-native yield infrastructure while keeping the core program insulated from adapter-specific assumptions.

5.5 Metaplex Core

RoundFi uses Metaplex Core for position assets.

Position NFTs represent membership rights and allow the Escape Valve flow, where a user can transfer/sell a position instead of defaulting.

6. Why the Programs Are Separated

The program split is intentional.

Design Choice	Reason
Core and reputation are separate	Reputation should eventually be consumable outside the pool application
Yield adapter is separate	Yield infrastructure is external and should be swappable
Position assets use Metaplex Core	Avoids building a custom NFT program and supports position transfer logic
SDK is separate from front-end	Enables app, indexer, and future partners to share encoders/decoders
Indexer is separate from programs	Keeps on-chain state minimal and exposes richer off-chain query surfaces

This makes RoundFi more modular and easier to audit.

7. Core Account Model

RoundFi uses deterministic PDAs to make state predictable and indexable.

7.1 ProtocolConfig

ProtocolConfig is the singleton protocol configuration account.

It stores:

- protocol authority;
- treasury;
- USDC mint;
- default yield adapter;
- reputation program;
- protocol fees;
- cycle fees;
- guarantee fund parameters;
- pause state;

- bump seed.

This account defines the global protocol parameters.

7.2 Pool

Pool is the central account for each group.

It stores:

- pool authority;
- seed ID;
- USDC mint;
- yield adapter;
- members target;
- installment amount;
- credit amount;
- total cycles;
- cycle duration;
- seed draw configuration;
- solidarity configuration;
- escrow release configuration;
- runtime status;
- current cycle;
- totals contributed and paid out;
- solidarity balance;
- escrow balance;
- yield accounting.

The pool account is the source of truth for lifecycle state.

7.3 Member

Member represents a user's position inside a pool.

It stores:

- pool reference;
- user wallet;
- position NFT asset;
- slot index;
- reputation level snapshot;
- stake basis points;
- deposited stake;
- contributions paid;
- total contributed;

- total received;
- escrow balance;
- on-time count;
- late count;
- default state;
- join timestamp.

This account links user behavior to a specific pool position.

7.4 ReputationProfile

ReputationProfile stores wallet-level reputation.

It can track:

- level;
- completed cycles;
- on-time payments;
- late payments;
- defaults;
- lifetime pools;
- score;
- cooldown timestamps;
- first seen / last updated timestamps.

This is the user-level behavioral identity.

7.5 Attestation

Attestation records a specific behavioral event.

Examples include:

- payment;
- late payment;
- default;
- cycle completion;
- level up.

Attestation records are intended to become compatible with a portable reputation model.

7.6 YieldVaultState

YieldVaultState tracks yield adapter state.

It can store:

- owner;
- principal;
- last harvest timestamp;

- accrued yield;
- bump seed.

This allows yield accounting to remain separate from pool lifecycle logic.

8. Vaults and Token Accounts

Each pool uses token accounts/vaults to keep capital separated.

Vault	Purpose
Pool USDC vault	Holds live contribution float and payout funds
Escrow vault	Holds locked escrow portions that vest over time
Solidarity vault	Holds solidarity contributions used in local protection logic
Yield vault	Holds funds routed to yield adapter infrastructure
Treasury ATA	Receives protocol fees

The separation of vaults matters because it reduces accounting ambiguity.

A default should not be able to drain unrelated reserves.

Yield infrastructure should not control the main pool vault directly.

9. Instruction Surface

The core protocol has a broad instruction surface covering lifecycle, accounting, security, and administration.

9.1 Protocol administration

Instruction	Purpose
initialize_protocol	Initializes global protocol configuration
update_protocol_config	Updates allowed protocol parameters
pause	Pauses user-facing instructions when needed
lock_treasury / treasury controls	Hardens treasury and governance paths

9.2 Pool lifecycle

Instruction	Purpose
create_pool	Creates a new savings group/pool
init_pool_vaults	Initializes pool token vaults
join_pool	Adds a user to a pool and creates their position
contribute	Processes a recurring payment
claim_payout	Lets a member receive their cycle payout
release_escrow	Releases vested escrow when conditions are met
close_pool	Finalizes a completed pool

9.3 Default and protection logic

Instruction	Purpose
settle_default	Processes a default path and applies protection logic
distribute_good_faith_bonus	Distributes solidarity/good-faith logic where applicable

9.4 Escape Valve

Instruction	Purpose
escape_valve_list	Lists a member position for sale
escape_valve_buy	Transfers the position to a buyer and re-anchors the member state

9.5 Yield routing

Instruction	Purpose
deposit_idle_to_yield	Routes idle capital into the yield adapter
harvest_yield	Harvests yield and applies the waterfall

The devnet record shows that the full M3 surface was exercised across multiple pools, including contribution, payout, yield, escape valve, close pool, and default settlement paths.

10. Pool State Machine

A pool moves through a controlled lifecycle.

```
Forming
  -> Active
  -> Completed
  -> Closed
```

Additional risk states may occur within the active period:

```
On-time cycle
  -> Late contribution
  -> Grace period
  -> Default settlement
```

The state machine must enforce:

- one member per position;
- correct cycle progression;
- contribution rules;
- payout eligibility;
- escrow locking;
- default handling;
- invariant preservation;
- safe pool closure.

The pool lifecycle is the foundation of the behavioral signal.

If the lifecycle is weak, the reputation is weak.

11. User Lifecycle

The user lifecycle is designed to produce measurable behavior.

11.1 Join

The user connects a wallet, joins a pool, posts required stake, and receives a position.

11.2 Pay

The user makes recurring monthly contributions.

Each successful contribution updates accounting and creates behavioral evidence.

11.3 Receive

At the assigned cycle, the user receives the payout/credit according to pool rules and reputation tier.

11.4 Continue

After receiving credit, the user continues paying future installments.

This is a key behavioral signal because it proves the user can continue obligations after receiving capital.

11.5 Graduate

If the user completes cycles successfully, reputation improves.

Higher reputation can reduce future collateral requirements.

11.6 Exit

If the user cannot continue, they may use the Escape Valve to sell the position instead of defaulting.

12. Reputation Architecture

The reputation architecture is the core product layer.

RoundFi's reputation is not meant to be an arbitrary score.

It is meant to be earned through behavior.

12.1 Reputation signals

The protocol can track:

- on-time payments;
- late payments;
- defaults;
- completed cycles;
- number of pools joined;
- time-based cooldowns;
- optional identity hints;
- level promotion.

12.2 Attestation model

Each behavioral event can become an attestation.

Attestations create a portable, inspectable record of user behavior.

This is the foundation for future B2B reputation queries.

12.3 Score model

The architecture supports a v1 score model with positive weights for good behavior and negative weights for late/default events.

The goal is not to create the final universal credit score immediately.

The goal is to produce clean, auditable, composable behavior data.

12.4 Reputation levels

RoundFi uses three levels:

Level	Label	Required stake	Capital access over stake
1	Beginner / Iniciante	50%	2x
2	Proven / Comprovado	30%	3.3x
3	Veteran / Veterano	10%	10x

This is the 50-30-10 rule.

The better the behavior, the lower the collateral.

13. Triple Shield Architecture

The Triple Shield is RoundFi's economic protection architecture.

It is a structural model, not a marketing slogan.

13.1 Shield 1 - Seed Draw

The Seed Draw keeps a large portion of early capital inside the pool instead of allowing the first cycle to drain the full amount immediately.

This creates a structural buffer at the beginning.

13.2 Shield 2 - Adaptive Escrow + Stake

The Adaptive Escrow system controls how much is released upfront and how much remains locked.

The stake requirement depends on reputation level.

Escrow releases are gated by continued payment behavior.

13.3 Shield 3 - Solidarity Vault + Yield Cascade

The Solidarity Vault collects a small share of contributions.

The Yield Cascade routes harvested yield through predefined buckets.

Together, they create additional buffers and economic support.

13.4 Narrative order vs. seizure order

The public narrative explains the shields by build order.

Technical default recovery may draw from buckets in a different sequence.

This distinction must remain clear in documentation.

14. Default Settlement

Default settlement is one of the most sensitive parts of the protocol.

The architecture must ensure that a defaulted member cannot drain the pool or block remaining members indefinitely.

The default path may involve:

- detecting late contribution state;
- enforcing grace period rules;
- marking the member as defaulted;
- applying local protection logic;
- drawing from appropriate buffers;
- preserving accounting invariants;
- writing a default record/attestation;
- allowing the pool to continue.

The correct claim is not that defaults cannot happen.

The correct claim is that the protocol is designed to bound loss and continue operating within modeled conditions.

15. Escape Valve Architecture

The Escape Valve allows a non-default exit.

A user who cannot continue can list the position for sale.

A buyer can purchase the position and continue the obligation.

The architecture uses a close-old/create-new member re-anchor pattern:

```
Seller position listed
  -> buyer pays seller
  -> old Member state is closed
  -> new Member state is created for buyer
  -> position asset is transferred
  -> pool accounting remains intact
```

The Escape Valve protects:

- the seller's reputation from default;
- pool continuity;
- buyer access to an existing position;

- protocol accounting.

It does not magically remove risk.

It gives users a market-based exit before default.

16. Yield Adapter Architecture

Yield is auxiliary to the core thesis.

RoundFi does not depend on yield to justify behavioral credit.

However, idle float can be routed through yield infrastructure to improve capital efficiency.

The yield adapter architecture follows a key principle:

Treat the adapter as untrusted.

The core program should:

- validate the expected adapter program ID;
- snapshot balances before and after CPI;
- use observed token deltas;
- isolate the yield vault from the pool vault;
- avoid trusting adapter-declared returns.

16.1 Canonical waterfall

The canonical waterfall is:

1. protocol fee;
2. Guarantee Fund top-up;
3. LP / Liquidity Angels share;
4. participant share.

16.2 Guarantee Fund

The Guarantee Fund is a reserve mechanism.

In v1, it acts mainly as payout-drain protection and reserve accounting.

Catastrophic draw paths are roadmap and should not be claimed as shipped.

17. Identity Layer

Identity is optional and modular.

The current architecture does not require KYC or proof-of-personhood to join.

Identity can become an additive signal for reputation confidence.

The design principle is:

Identity can enrich reputation, but behavior remains the primary signal.

Possible identity providers may include Civic or future providers.

The important architectural decision is that identity is not a hard gate in the current phase.

18. Anti-Gaming Architecture

Reputation systems attract farming.

RoundFi's architecture tries to make high-quality reputation expensive to fake.

Anti-gaming mechanisms include:

- recurring capital commitment;
- time requirements;
- payment cycles;
- cooldowns;
- same-issuer / same-subject rate limits;
- negative default records;
- default stickiness;
- identity weighting hints;
- economic cost of sustaining fake wallets.

The architecture does not claim to eliminate sybil risk completely.

It raises the cost of producing credible behavior.

19. SDK Architecture

The TypeScript SDK is a critical bridge between the application, indexer, and on-chain programs.

The current SDK includes:

- action helpers;
- account decoders;
- PDA derivation utilities;
- event helpers;
- IDL-free encoders used by front-end and indexer surfaces.

The SDK reduces duplicate logic and helps keep app-to-chain behavior consistent.

It is also the foundation for future partner integrations.

20. Front-End Architecture

The front-end is a Next.js application with Solana wallet integration.

It includes routes such as:

- public landing;
- home dashboard;

- wallet/carteira;
- groups/grupos;
- reputation/reputacao;
- market/mercado;
- insights;
- Stress Lab;
- Demo Studio/admin;
- developer demo.

The app is not only a static mock.

It includes wallet flows, session state, reactive score mechanics, modals, devnet interactions for selected actions, and Stress Lab visualization.

The front-end's role is to demonstrate the user and protocol experience clearly enough for hackathon evaluation and technical review.

21. Indexer and B2B API Architecture

The indexer is the foundation for the future B2B oracle.

The current scaffold includes:

- Fastify service;
- Helius webhook integration path;
- Prisma/Postgres schema;
- event store design;
- getProgramAccounts backfill logic;
- shared decoders from the SDK.

The production indexer is not complete yet.

The long-term goal is:

```
On-chain events and attestations
  -> indexer reconciliation
  -> canonical reputation profile
  -> B2B score API
  -> protocols / fintechs / neobanks query before credit decisions
```

This is Phase 3 of the business model.

22. Stress Lab Architecture

The Stress Lab is a pure TypeScript economic simulator.

It models:

- pool configurations;
- contribution cycles;
- defaults;

- post-default behavior;
- cascade conditions;
- triple-default scenarios;
- yield assumptions;
- capital structure;
- net solvency;
- Triple Shield behavior.

The Stress Lab acts as the L1 reference model for economic logic.

The architecture's goal is to compare simulator behavior with on-chain behavior through parity tests.

The honest claim is:

RoundFi remains solvent within modeled conditions.

Not:

RoundFi can never fail.

23. Devnet Architecture and Evidence

The devnet deployment record shows:

- four program surfaces deployed;
- three pools exercised;
- contribution paths;
- payout paths;
- yield paths;
- escape valve paths;
- close pool path;
- default settlement path;
- browser-signed contribute() transaction;
- browser-signed claim_payout() transaction;
- Solscan-verifiable evidence;
- mpl-core issue discovered and fixed;
- Triple Shield guards observed on real funds.

This is important because it shows execution beyond a front-end demo.

The project is not only a pitch.

It has devnet evidence.

24. Security Architecture

RoundFi's security architecture focuses on protecting funds, preserving accounting, and preventing invalid state transitions.

Key areas include:

- PDA seed correctness;
- signer and authority validation;
- vault separation;
- CPI program validation;
- token account balance checks;
- escrow gates;
- default settlement invariants;
- NFT transfer correctness;
- yield adapter assumptions;
- pause and treasury protections;
- event and attestation correctness;
- test coverage;
- internal audit documentation.

The project includes an internal self-audit and responsible disclosure policy.

External audit is still required before mainnet.

25. Shipped, Under Verification, Roadmap

25.1 Shipped / Demoable Today

- live front-end prototype;
- Solana Devnet deployment;
- four program surfaces;
- three devnet pools;
- Stress Lab;
- selected app-to-chain flows;
- open-source repository;
- SDK foundation;
- indexer scaffold;
- internal audit documentation;
- Solscan evidence.

25.2 Implemented / Under Verification

- expanded protocol surface;
- bankrun integration coverage;
- L1-to-L2 parity beyond Healthy canary;
- reputation lifecycle verification;
- indexer-backed reads;

- deeper production-style tests.

25.3 Roadmap

- production SAS issuance;
- hosted production indexer;
- public B2B reputation API;
- protocol partner integrations;
- external audit;
- mainnet beta;
- compliance/legal structuring;
- larger-scale pilots.

26. Architecture Risks

RoundFi's architecture has meaningful risks.

Risk	Description	Mitigation Direction
Smart contract risk	Financial logic can fail if state transitions or accounting are wrong	External audit, tests, invariants, phased mainnet
Reputation gaming	Users may farm score with artificial wallets	Time, capital cost, cooldowns, identity hints, default penalties
SAS dependency	Attestation infrastructure may change or remain early	Abstract attestation interface and maintain internal compatibility layer
Yield adapter risk	External yield source may underperform or behave unexpectedly	Treat adapter as untrusted, isolate vaults, use observed balances
Indexer correctness	B2B API depends on accurate event reconciliation	Use deterministic PDAs, shared SDK decoders, reconciliation tests
Regulatory risk	Credit/pooling/reputation may be regulated	Legal review before production
Adoption risk	Protocols may not trust the score initially	Pilot integrations and transparent evidence

27. Design Principles for Future Development

RoundFi should continue following these architecture principles:

1. Keep behavior verifiable.

Reputation must come from actions, not self-declared profiles.

2. Keep state deterministic.

Use PDAs and predictable account layouts.

3. Keep adapters swappable.

External yield and attestation dependencies should not trap the core.

4. Keep claims honest.

Devnet is not mainnet. Simulated solvency is not guaranteed solvency.

5. Keep user incentives clear.

The user benefit is lower collateral and better credit access.

6. Keep Phase 3 in mind.

Every behavior record should eventually support a reputation oracle.

7. Keep security ahead of growth.

No mainnet credit without audit, limits, monitoring, and legal review.

28. Repository Mapping

The repository is organized around on-chain programs, SDK, front-end, scripts, tests, grant materials, and documentation.

Path	Purpose
programs/roundfi-core/	Core Anchor program and pool state machine
programs/roundfi-reputation/	Reputation and attestation program
programs/roundfi-yield-mock/	Devnet yield adapter
programs/roundfi-yield-kamino/	Kamino-oriented yield adapter
sdk/	TypeScript SDK, decoders, PDA helpers, Stress Lab
services/indexer/	Indexer and future B2B API foundation
app/	Next.js front-end and demo application
scripts/	Devnet deployment and testing scripts
tests/	Integration, parity, security, lifecycle, and edge tests
docs/	Architecture, status, deployment, pitch, and strategic docs
grant/	Grant and hackathon supporting materials

29. Evaluation Readiness

For a hackathon evaluator, the architecture demonstrates:

- actual on-chain deployment;
- technical complexity beyond a static app;
- Solana-native execution;
- composability with ecosystem components;
- clear separation between current proof and future roadmap;
- transparent status tracking;
- internal security awareness;
- a credible path toward a B2B reputation oracle.

The strongest technical evidence is not one single feature.

It is the combination of:

- devnet programs;
- front-end user flows;
- SDK wiring;
- Stress Lab simulator;
- Solscan-verifiable transactions;
- internal audit;

- clear docs.

30. Conclusion

RoundFi's architecture is designed to create a new credit primitive for Solana.

The core protocol generates behavior.

The reputation program converts that behavior into portable identity.

The collateral ladder turns reputation into capital efficiency.

The Stress Lab tests the economic model.

The indexer and API roadmap turn the reputation layer into a future B2B product.

RoundFi is still hackathon-stage and not yet a production mainnet credit system.

But the architecture is coherent:

```
payments -> attestations -> reputation -> lower collateral -> reputation oracle
```

That is the architectural foundation of RoundFi.

31. One-Sentence Summary

RoundFi is a Solana-native behavioral credit architecture that turns recurring payments into portable reputation and uses that reputation to reduce collateral requirements over time.

32. Related Documents

- 01-roundfi-overview.md
- 02-technical-whitepaper.md
- 04-behavioral-reputation-score.md
- 05-stress-lab-economic-model.md
- 06-market-and-gtm.md
- 07-business-model-b2b-oracle.md
- 08-competitive-analysis.md
- 09-risk-and-compliance.md
- 10-user-guide.md
- 11-devnet-status-and-proof.md

33. References

RoundFi repository documents

- README.md - project thesis, current repo status, development status, core mechanics.
- docs/status.md - shipped vs. roadmap status register.

- docs/devnet-deployment.md - devnet program IDs, deployment records, Solscan evidence.
- docs/architecture.md - original architecture specification and account model.
- docs/pitch-alignment.md - reputation ladder, Triple Shield mapping, solvency framing.
- docs/yield-and-guarantee-fund.md - yield adapter, waterfall, Guarantee Fund details.
- docs/security/self-audit.md - internal audit and threat model.
- sdk/src/stressLab.ts - Stress Lab reference simulator.
- programs/roundfi-core/ - core Anchor program.
- programs/roundfi-reputation/ - reputation and attestation program.

Ecosystem references

- Solana documentation.
- Anchor framework documentation.
- Solana Attestation Service documentation.
- Kamino Finance documentation.
- Switchboard documentation.
- Metaplex Core documentation.

Disclaimer

This document is for technical and strategic discussion only. It is not legal, financial, investment, tax, or regulatory advice. RoundFi is currently a hackathon-stage protocol and should not be treated as a production credit product until external audit, legal review, mainnet risk controls, and partner validation are complete.